

Assignment -7.5

Hall Ticket: 2303A510i2

Name: K.Rohith

Batch -23

Task 1 (Mutable Default Argument – Function Bug)

Task: Analyze given code where a mutable default argument causes unexpected behavior. Use AI to fix it.

Bug: Mutable default argument

```
def add_item(item, items=[]):
```

```
    items.append(item)
```

```
    return items
```

```
print(add_item(1))
```

```
print(add_item(2))
```

Expected Output: Corrected function avoids shared list bug.

Code:

```
1  #error Dont use mutable default argument
2  def add_item(item, items=None):
3      if items is None:
4          items = []
5          items.append(item)
6          return items
7
8  print(add_item(1, [10, 20]))
9  print(add_item(2, [11.11, 22.22]))
```

Output:

```
on3.11.exe C:/Users/rohit/OneDrive/Desktop/ai/week 7.py
PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/rohit/OneDrive/Desktop/ai/week 7.py"
[10, 20, 1]
[11.11, 22.22, 2]
PS C:\Users\rohit\OneDrive\Desktop\ai>
```

Task 2 (Floating-Point Precision Error)

Task: Analyze given code where floating-point comparison fails. Use AI to correct with tolerance.

Bug: Floating point precision issue

```
def check_sum():
```

```
return (0.1 + 0.2) == 0.3
```

```
print(check_sum())
```

Expected Output: Corrected function

Code:

```
8 #error Floating-point precision
9 ✓ def check_sum():
10     return abs((0.1 + 0.2) - 0.3) < 1e-10
11
12 print(check_sum())
13
```

Output:

```
PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/rohit/OneDrive/Desktop/ai/week 7.py"
True
PS C:\Users\rohit\OneDrive\Desktop\ai>
```

Task 3 (Recursion Error – Missing Base Case)

Task: Analyze given code where recursion runs infinitely due to missing base case. Use AI to fix.

Bug: No base case

```
def countdown(n):
```

```
    print(n)
```

```
    return countdown(n-1)
```

```
countdown(5)
```

Expected Output : Correct recursion with stopping condition.

Code:

```
15 #Error Recursion limit exceeded
16 def countdown(n):
17     if n == 0:
18         return
19     print(n)
20     countdown(n - 1)
21
22 countdown(5)
23 |
```

Output:

```
● PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/rohit/OneDrive/Desktop/ai/week 7.py"
5
4
3
2
1
○ PS C:\Users\rohit\OneDrive\Desktop\ai>
```

Task 4 (Dictionary Key Error)

Task: Analyze given code where a missing dictionary key causes error. Use AI to fix it.

Bug: Accessing non-existing key

```
def get_value():
    data = {"a": 1, "b": 2}
    return data["c"]
print(get_value())
```

Expected Output: Corrected with .get() or error handling.

Code:

```

25 #Error KeyError when accessing dictionary
26 # with a non-existent key
27 def get_value():
28     data = {"a": 1, "b": 2}
29     return data.get("c", "Key not found")
30
31 print(get_value())
32

```

Output

```

PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/rohit/OneDrive/Desktop/ai/week 7.py"
Key not found
PS C:\Users\rohit\OneDrive\Desktop\ai>

```

Task 5 (Infinite Loop – Wrong Condition)

Task: Analyze given code where loop never ends. Use AI to detect and fix it.

Bug: Infinite loop

```
def loop_example():
```

```
    i = 0
```

```
    while i < 5:
```

```
        print(i)
```

Expected Output: Corrected loop increments i.

Code:

```

34 #Error in loop due to incorrect indentation
35 def loop_example():
36     i = 0
37     while i < 5:
38         print(i)
39         i += 1
40
41 loop_example()
42

```

Output:

```
PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/rohit/OneDrive/Desktop/ai/week 7.py"
0
1
2
3
4
PS C:\Users\rohit\OneDrive\Desktop\ai>
```

Task 6 (Unpacking Error – Wrong Variables)

Task: Analyze given code where tuple unpacking fails. Use AI to fix it.

Bug: Wrong unpacking

a, b = (1, 2, 3)

Expected Output: Correct unpacking or using _ for extra values.

```
45 # Error in unpacking values from a tuple
46 # using underscore for unused variable
47 a, b, _ = (1, 2, 3)
48 print(a, b)
49 |
```

OUTPUT:

```
PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/rohit/OneDrive/Desktop/ai/week 7.py"
1 2
PS C:\Users\rohit\OneDrive\Desktop\ai>
```

Ln 49, Col 1 Spaces: 4 UTF-8 CRLF {} Python 3.11.9 (Microsoft Store)

Task 7 (Mixed Indentation – Tabs vs Spaces)

Task: Analyze given code where mixed indentation breaks execution. Use AI to fix it.

Bug: Mixed indentation

```
def func():
```

```
    x = 5
```

```
    y = 10
```

```
    return x+y
```

Expected Output : Consistent indentation applied.

Code:

```
51  # Bug: Mixed indentation
52  def func():
53      x = 5
54      y = 10
55      return x + y
56
57  print(func())
58
```

Output:

```
PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:
15
PS C:\Users\rohit\OneDrive\Desktop\ai>
```

Task 8 (Import Error – Wrong Module Usage)

Task: Analyze given code with incorrect import. Use AI to fix.

Bug: Wrong import

```
import maths
```

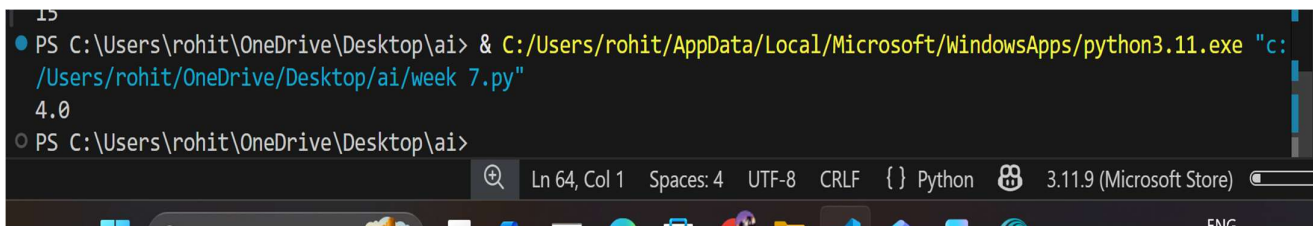
```
print(maths.sqrt(16))
```

Expected Output: Corrected to import math

Code:

```
60 # Correct import
61 import math as m
62
63 print(m.sqrt(16))
64
```

OutPut:



```
15
PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/rohit/OneDrive/Desktop/ai/week 7.py"
4.0
PS C:\Users\rohit\OneDrive\Desktop\ai>
```

Ln 64, Col 1 Spaces: 4 UTF-8 CRLF { } Python 3.11.9 (Microsoft Store)

Task 9 (Unreachable Code – Return Inside Loop)

Task: Analyze given code where a return inside a loop prevents full iteration. Use AI to fix it.

Bug: Early return inside loop

```
def total(numbers):
```

```
    for n in numbers:
```

```
        return n
```

```
print(total([1,2,3]))
```

Expected Output: Corrected code accumulates sum and returns after loop.

Code:

```

66 # Task 9 (Logical Error – Incorrect Loop Logic)
67 def total(numbers):
68     total_sum = 0
69     for n in numbers:
70         total_sum += n
71     return total_sum
72
73 print(total([1, 2, 3]))
74

```

Output:

```

PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe "C:/Users/rohit/OneDrive/Desktop/ai/week 7.py"
6
PS C:\Users\rohit\OneDrive\Desktop\ai>

```

Task 10 (Name Error – Undefined Variable)

Task: Analyze given code where a variable is used before being defined. Let AI detect and fix the error.

Bug: Using undefined variable

```

def calculate_area():
    return length * width
print(calculate_area())

```

Requirements:

- Run the code to observe the error.
- Ask AI to identify the missing variable definition.
- Fix the bug by defining length and width as parameters.
- Add 3 assert test cases for correctness.

Expected Output :

- Corrected code with parameters.
- AI explanation of the bug.

Successful execution of assertions.

Code:


```

75 # Bug: Using undefined variable
76 def calculate_area():
77     radius = 5
78     area = 3.14 * radius * radius
79     return area
80
81 print(calculate_area())
82

```

OutPut:

```

● PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:
/Users/rohit/OneDrive/Desktop/ai/week 7.py"
78.5
○ PS C:\Users\rohit\OneDrive\Desktop\ai>

```

Task 11 (Type Error – Mixing Data Types Incorrectly)

Task: Analyze given code where integers and strings are added incorrectly. Let AI detect and fix the error.

Bug: Adding integer and string

```

def add_values():
    return 5 + "10"
print(add_values())

```

Requirements:

- Run the code to observe the error.
- AI should explain why int + str is invalid.
- Fix the code by type conversion (e.g., int("10") or str(5)).
- Verify with 3 assert cases.

Expected Output #6:

- Corrected code with type handling.
- AI explanation of the fix. Successful test validation.

Code:

```

81 # print(calculate_area())
82
83
84 def add_values():
85     return 5 + int("10")
86
87 print(add_values())
88
89 # Test validation
90 assert add_values() == 15
91 assert 7 + int("3") == 10
92 assert int("20") + 5 == 25
93 print("All tests passed!")
94

```

OutPut:

```

PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/rohit/OneDrive/Desktop/ai/week 7.py"
15
All tests passed!
PS C:\Users\rohit\OneDrive\Desktop\ai>

```

Task 12 (Type Error – String + List Concatenation)

Task: Analyze code where a string is incorrectly added to a list.

Bug: Adding string and list

```

def combine():
    return "Numbers: " + [1, 2, 3]

print(combine())

```

Requirements:

- Run the code to observe the error.
- Explain why str + list is invalid.
- Fix using conversion (str([1,2,3]) or " ".join()).
- Verify with 3 assert cases.

Expected Output:

- Corrected code
- Explanation
- Successful test validation

```

96  def combine():
97      return "Numbers: " + str([1, 2, 3])
98
99      print(combine())
100
101      assert combine() == "Numbers: [1, 2, 3]"
102      assert "List: " + str([4, 5]) == "List: [4, 5]"
103      assert "Values: " + str([]) == "Values: []"
104      print("All tests passed!")

```

OutPut:

```

PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/rohit/OneDrive/Desktop/ai/week 7.py"
Numbers: [1, 2, 3]
PS C:\Users\rohit\OneDrive\Desktop\ai>

```

Task 13 (Type Error – Multiplying String by Float)

Task: Detect and fix code where a string is multiplied by a float.

Bug: Multiplying string by float

```

def repeat_text():
    return "Hello" * 2.5
print(repeat_text())

```

Requirements:

- Observe the error.
- Explain why float multiplication is invalid for strings.
- Fix by converting float to int.
- Add 3 assert test cases.

Code:

```

106     def repeat_text():
107         return "Hello" * int(2.5)
108
109     print(repeat_text())
110
111     assert repeat_text() == "HelloHello"
112     assert "Hi" * int(3.0) == "HiHiHi"
113     assert "A" * int(1.9) == "A"
114     print("ALL tests passed!")

```

Output:

```

PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/rohit/OneDrive/Desktop/ai/week 7.py"
HelloHello
PS C:\Users\rohit\OneDrive\Desktop\ai>

```

Task 14 (Type Error – Adding None to Integer)

Task: Analyze code where None is added to an integer.

Bug: Adding None and integer

```

def compute():
    value = None
    return value + 10
print(compute())

```

Requirements:

- Run and identify the error.
- Explain why NoneType cannot be added.
- Fix by assigning a default value.
- Validate using asserts.

Code

```
117 def compute():
118     value = None
119     if value is None:
120         value = 0
121     return value + 10
122
123 print(compute())
124
125 assert compute() == 10
126 assert (0 + 10) == 10
127 assert (5 + 10) == 15
128
```

Output:

```
PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/rohit/OneDrive/Desktop/ai/week 7.py"
10
PS C:\Users\rohit\OneDrive\Desktop\ai>
```

Task 15 (Type Error – Input Treated as String Instead of Number)

Task: Fix code where user input is not converted properly.

Bug: Input remains string

```
def sum_two_numbers():
a = input("Enter first number: ")
b = input("Enter second number: ")
return a + b
print(sum_two_numbers())
```

Requirements:

- Explain why input is always string.
- Fix using `int()` conversion.
- Verify with assert test cases.

CODE:

```
130 def sum_two_numbers(a, b):
131     return int(a) + int(b)
132
133 print(sum_two_numbers("2", "3"))
134
135 assert sum_two_numbers("2", "3") == 5
136 assert sum_two_numbers("10", "5") == 15
137 assert sum_two_numbers("7", "8") == 15
138
```

OUTPUT:

```
● PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:
/Users/rohit/OneDrive/Desktop/ai/week 7.py"
5
○ PS C:\Users\rohit\OneDrive\Desktop\ai>
```